

Pruebas

Álvaro González Sotillo

18 de marzo de 2026

Índice

1. Pruebas del <i>software</i>	1
2. Cobertura de código	3
3. Referencias	4

1. Pruebas del *software*

- El desarrollo solo termina tras comprobar que el código funciona
 - Un método
 - Varias clases que colaboran
 - Una aplicación
 - Un conjunto de aplicaciones y servidores
- Se comprueba que funciona mediante **pruebas**

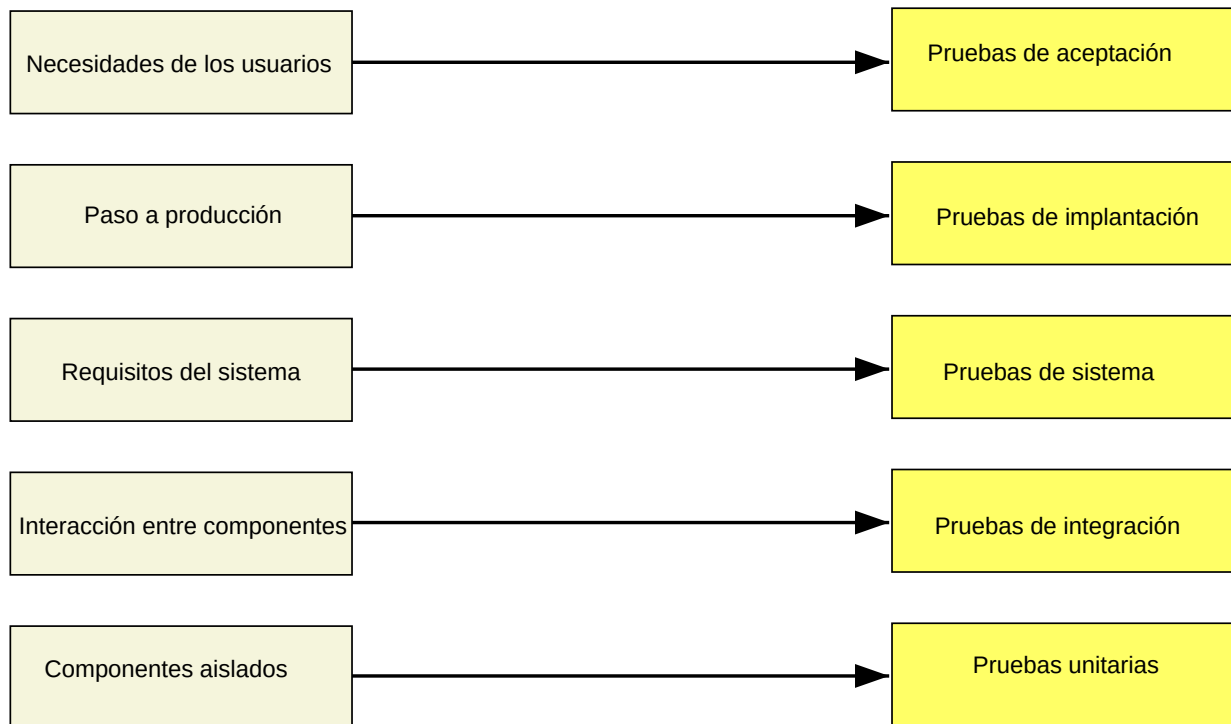
1.1. Ejemplos

Objeto de prueba	Caso de prueba
<code>Math.abs()</code> se corresponde con el valor absoluto	Se espera que <code>Math.abs(-10)</code> sea 10
Si Mario cae sobre un <i>goompa</i> , Mario rebota	Se espera que Mario rebote al saltar sobre el pimer <i>goompa</i> en el mundo
No se puede comprar a crédito	Se espera que el usuario US9474, con saldo 35, no puede comprar el art

1.2. Qué es un caso de prueba

- Objeto de prueba:
 - qué requisito se está comprobando
- Condiciones de la prueba:
 - estado del sistema previo a la prueba: variables, base de datos, conexiones, ventanas abiertas, ficheros...
 - acciones sobre el sistema: invocar métodos, pulsar botones, lanzar comandos...
- Resultado esperado
 - estado final del sistema: variables, base de datos, ventanas mostradas...

1.3. Nivel de las pruebas



1.3.1. Pruebas unitarias

- Prueban unidades de código
- En java, un método o una clase

1.3.2. Pruebas de integración

- Comprueban varias clases trabajando de forma conjunta

1.3.3. Pruebas de sistema

- Se verifica el sistema completo (aplicación o conjunto de aplicaciones)
- Se suelen realizar en sistemas no finales
 - Entorno de preproducción / entorno de pruebas

1.3.4. Pruebas de implantación

- Se verifica el sistema completo (como las pruebas de sistema)
- Pero el entorno es el definitivo
 - Entorno de producción

1.3.5. Pruebas de aceptación

- Se verifica el sistema completo (como las pruebas de implantación)
- Se realizan por el usuario final
- Suelen tener valor contractual: si las pruebas se aceptan, el proyecto está legalmente entregado y es funcional

1.4. Cuánto probar

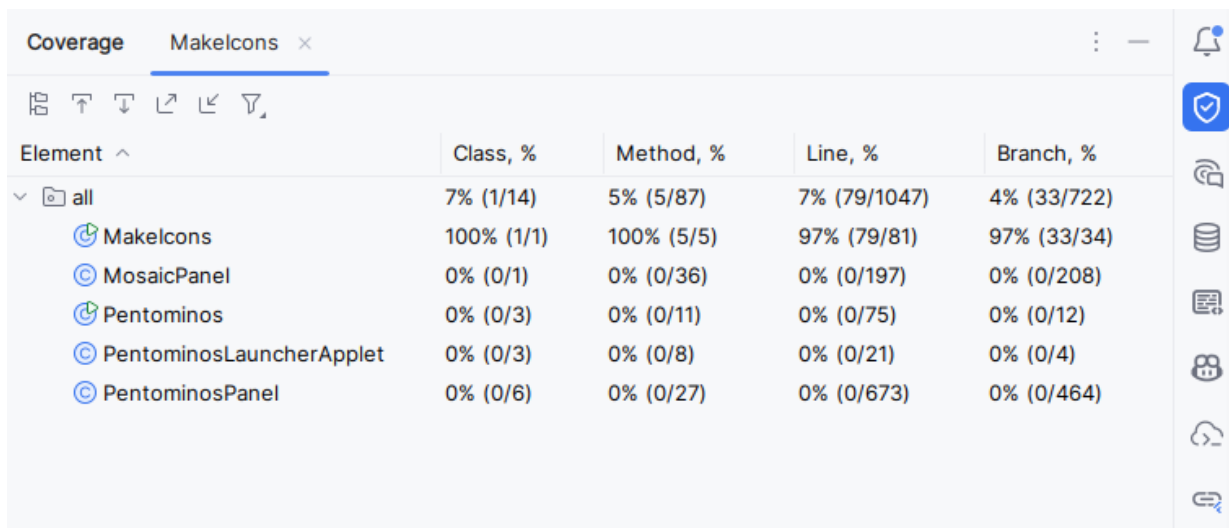
- ¿Cuántas pruebas se necesitan para asegurar que `Math.abs()` es correcto?
- Que funcione para 4, 24, -2341, 3113 no garantiza al 100 % que funcione para -54.
- Pero tampoco es práctico probar para todos los números `int`, `log`, `float` y `double`
- Es necesario definir un conjunto de pruebas
 - Factible: no demasiadas pruebas
 - Completo: lo probado garantiza que los casos no probados también son correctos

2. Cobertura de código

- Idea: el código no puede estar probado si no se pasa por todas las líneas de código
- Pruebas de cobertura: durante las pruebas, todas las líneas de código se deben ejecutar al menos una vez

2.1. Cobertura en IntelliJ

- SHIFT SHIFT run with coverage

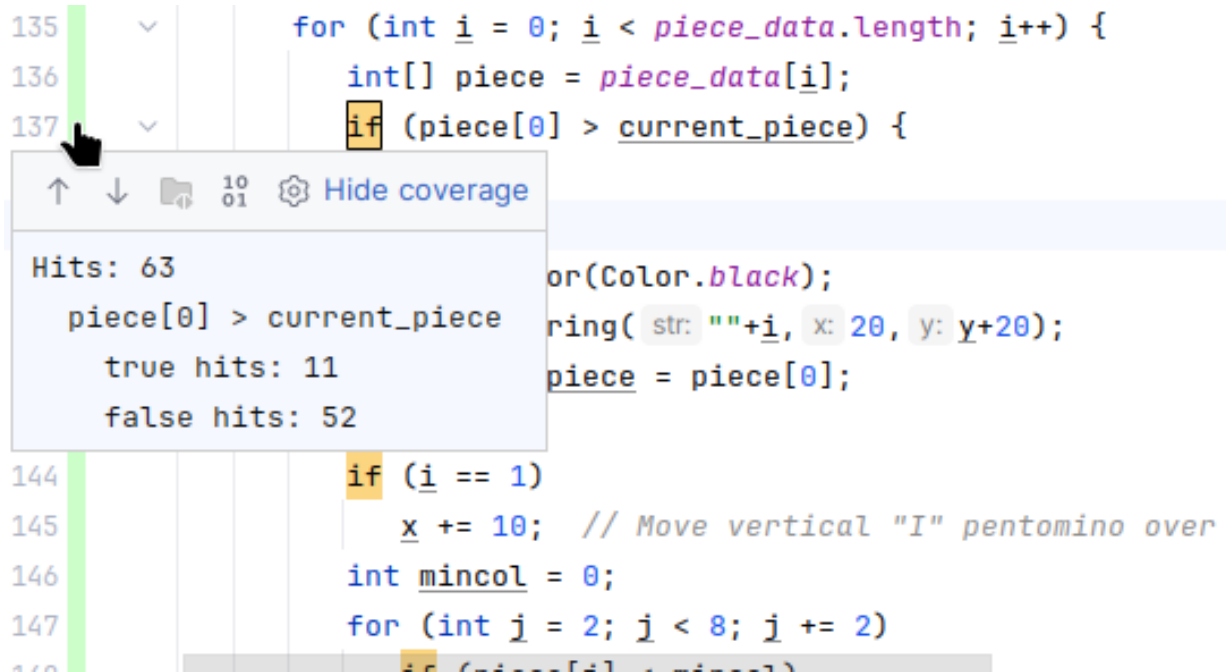


The screenshot shows the IntelliJ Coverage tool window. The title bar says 'Coverage' and 'Makelcons'. Below the title bar is a toolbar with icons for coverage-related actions. The main table displays coverage data for various elements. The columns are 'Element', 'Class, %', 'Method, %', 'Line, %', and 'Branch, %'. The data is as follows:

Element ^	Class, %	Method, %	Line, %	Branch, %
all	7% (1/14)	5% (5/87)	7% (79/1047)	4% (33/722)
Makelcons	100% (1/1)	100% (5/5)	97% (79/81)	97% (33/34)
MosaicPanel	0% (0/1)	0% (0/36)	0% (0/197)	0% (0/208)
Pentominos	0% (0/3)	0% (0/11)	0% (0/75)	0% (0/12)
PentominosLauncherApplet	0% (0/3)	0% (0/8)	0% (0/21)	0% (0/4)
PentominosPanel	0% (0/6)	0% (0/27)	0% (0/673)	0% (0/464)

2.2. Condiciones y decisiones

- Cobertura de decisiones: cada salto (`if`, `while`, `for`) se ha ejecutado como cierto y falso
- Cobertura de condiciones: cada parte de una condición compuesta (`and`, `or`) se ha ejecutado como cierta y falso
 - Esta es más complicada de conseguir



2.3. Cobertura en proyectos grandes

- Por ejemplo, [Jacoco](#)
- Se puede [integrar en SonarQube](#)

3. Referencias

- Formatos:
 - [Transparencias](#)
 - [PDF](#)
 - [Página web](#)
 - [EPUB](#)
- Alojado en [Github](#)
- <https://entornos.abrilcode.com/doku.php?id=apuntes:pruebas>